esprit Se former autrement	**Atelier Sauvegarde et chargement des données du jeu**	
	Matière : Projet 1A Classe(s) : 1A Equipe : Projet 1A	Unité pédagogique : Algorithmique & Programmation

Introduction

La « sauvegarde» est une fonctionnalité proposée par certains jeux vidéo pour permettre au joueur, par une simple pression sur une touche prédéfinie du clavier, de sauvegarder son statut et sa progression dans le niveau. Il est accompagné d'une fonctionnalité de “Chargement” qui permet de charger le dernier point sauvegardé.

L’objectif de cet atelier est de vous présenter un algorithme permettant de sauvegarder et charger les données du jeu.

1- Sauvegarde du jeu

La signification de "sauvegarder le jeu" peut varier considérablement d'un jeu à l'autre, mais l'idée générale de permettre aux joueurs de quitter le jeu et de reprendre là où ils l'ont laissé plus tard. En fonction du type de jeu que vous créez, vous aurez peut-être besoin que de quelques informations de base, telles que la position du background ainsi que la position atteinte par le joueur et peut-être le score et le nombre de vie.

Pour ce faire il faut passer par deux phases:

- Phase 1 : Mise en place d’un sous menu pour la sauvegarde
- Phase 2: Implémentation de fonctions de sauvegarde.

1.1 Mettre en place un sous menu de sauvegarde

La figure 1, vous montre un exemple de sous menu flottant sur le background du jeu permettant de proposer l’option de sauvegarde du jeu ou de revenir vers le jeu et cette suite à un appui sur le bouton échapper.

Figure 1 : Sous menu de sauvegarde

Pour mettre en place ce type de menu, nous allons utiliser les ressources suivantes:

Figure 2 : Images du Menu

Nous allons faire des modifications au niveau du module (Menu.h/Menu.c) que nous avons déjà utilisé pour mettre en place le menu du jeu. Une définition possible de la structure Menu peut être la suivante:

```
struct Boutton
{
   SDL_Surface* Img[2];
   SDL_Rect Pos;
   Mix_Chunk *son;
   int num; // indiquant l'ordre du bouton dans le menu
};
typedef struct Boutton Boutton;
struct Menu
{
   SDL_Surface * image_bg;
   SDL_Rect pos_bg;
   int nbr_boutton; // indiquant le nombre de bouton du menu
   Boutton tab[10];
   Mix_Music *musique;
   int num_bt_actif; // indiquant le numéro du bouton actuellement choisi
};
typedef struct Menu Menu;
```

Nous allons rajouter alors les fonctions suivantes :

1. La fonction ***init_SousMenu*** qui retourne une variable de type ***SousMenu*** et permettant de:

    a. Charger l'image du background du sous menu
    b. Charger les 4 images pour les deux boutons comme illustré sur la figure 2.
    c. Charger le son bref qui sera lancé lorsque nous appuyons sur un bouton
    d. Charger le son continu accompagnant le sous menu
    e. Initialiser la position du background
    f. Initialiser la position de chaque bouton
    g. Indiquer le numéro de chaque bouton
    h. initialiser le premier bouton comme bouton actif

2. La fonction ***afficher_SousMenu*** qui prend en paramètre le sous menu et l'écran et permettant

de:

a. Afficher le background
b. Pour chaque bouton
    i. si le numéro de bouton actif est égale au numéro de bouton alors
        - afficher l'image représentant le bouton dans son cas actif
    ii. sinon
        - afficher l'image représentant le bouton dans son cas non actif

3. La fonction ***update_SousMenu*** (pour rendre les boutons sélectionnées à chaque fois qu'on appuis sur les flèches haut et bas vous pouvez aussi faire ce traitement directement dans le main)

   Cette fonction prend en paramètre le sous menu, une variable d'état ***input***[1] représentant l'événement clavier fait par le joueur et retourne le sous menu et une variable ***interface*** indiquant l'action à faire ensuite[2] permettant de:

   a. Selon la valeur de la variable input, si ***input*** reprend la flèche bas :
      - Incrémenter le numéro de bouton actif par 1. N.B: il ne faut pas que la variable indiquant le numéro de bouton actif dépasse le nombre total des boutons.
   b. Si ***input*** reprend la flèche haut
      - Décrémenter le numéro de bouton actif par 1. N.B: il ne faut pas que la variable indiquant le numéro de bouton actif soit inférieure à 1.
   c. Si ***input*** reprend la le bouton entrée
      - Retourner soit le numéro d'interface adéquat à l'action à faire par le bouton (l'interface du jeu) bien une autre valeur indiquant qu'il faut lancer la sauvegarde du jeu (-1 par exemple).

4. La fonction **liberer_SousMenu** permettant de libérer toutes les images et son du sous menu.

Nous modifions aussi le main du jeu pour prendre en considération les nouvelles fonctionnalités. Ainsi, nous devons tout d'abord définir une variable de type menu. Ensuite, nous allons appeler la fonction ***init_SousMenu*** avant la boucle du jeu.

Comme nous l'avons expliqué au niveau du guide de conception, une fois qu'on est dans la boucle du jeu la première chose à faire est l'affichage. Alors pour prendre en considération l'affichage du sous menu, nous pouvons proposer l'algorithme suivant:

- si interface == 1
    a. afficher menu du jeu
- si interface == 2
    a. afficher menu des options
- si interface ==3
    a. afficher le jeu
- si interface == 4

[1] La variable **input** peut être déclarée de type entier dans le main du jeu et permet de définir l'événement fait par le joueur. Cette variable sera remplie lors de la fonction input. Par exemple,si le joueur appui sur la flèche haut la variable input prendra la valeur 1, et ,si le joueur appui sur la flèche bas la variable input prendra la valeur 2 ainsi de suite.

[2] L'action à faire ensuite peut être définie aussi à travers une variable définie dans le main, nous pouvons déclarer une variable **interface** indiquant l'interface à afficher. Par exemple , on met la variable à 0 pour dire qu'on veut afficher le menu principal du jeu, 2 pour afficher le menu des options , 3 pour le premier niveau du jeu , 4 pour le sous menu de sauvegarde, 5 pour le premier énigme ainsi de suite.

a. afficher le jeu
b. afficher le sous menu de sauvegarde

La fonction ***update_SousMenu*** sera ainsi appelée à la fin du boucle de jeu et la fonction **liberer_SousMenu** à la fin du programme principal.

1.2 Implémentation de fonctions de sauvegarde

Afin de retrouver le même état du jeu après la sauvegarde, nous avons intérêt à sauvegarder :

- la position du joueur
- la position de caméra (le dernier point atteint par le joueur sur background)
- Le score, nombre de vie du joueur et le temps
- la liste des ennemies , obstacle ou entité secondaire restant dans le jeu (ceux qui ne sont pas encore tués ou consommés par le joueur) ainsi que leurs positions
- les états des énigmes (résolus ou non)

Alors pour ce faire nous devons faire recours aux fichiers textes ou binaires. Nous proposons de développer une fonction **sauvegarde_jeu** au niveau du main par exemple et qui suit l'algorithme suivant:

1. Ouvrir le fichier en mode écriture
2. vérifier la bonne ouverture du fichier
3. écrire les informations nécessaires avec un format simple
4. Fermer le fichier

L'appel de cette fonction doit être fait au niveau du main aussi lorsque la valeur de retour de la fonction ***update_SousMenu*** est -1 (la valeur définie pour lancer la sauvegarde, voir sous section précédente).

2- Le chargement du jeu

Le chargement du jeu représente l'opération inverse de l'opération de sauvegarde. Pour la mettre en place nous devons passer aussi par deux phases :

- L'implémentation des fonctions de chargement
- La mise à jour du menu pour ajouter le bouton " Load game ".

2.1 Implémentation de fonctions de chargement

Nous allons commencer par l'implémentation de la fonction **load_game** ayant comme objectif de lire le fichier créé lors de l'étape de sauvegarde. Cette fonction doit retourner toutes les variables préalablement enregistrées. Nous proposons l'algorithme suivant:

5. Ouvrir le fichier en mode lecture<
6. vérifier la bonne ouverture du fichier
7. lire les informations nécessaires et les enregistrer dans les variables de retour.
8. Fermer le fichier

2.2 Mise à jour du menu

La figure 3 vous montre un exemple de menu principale avec le bouton “ load “ permettant de reprendre l’état du dernier sauvegarde du jeu.

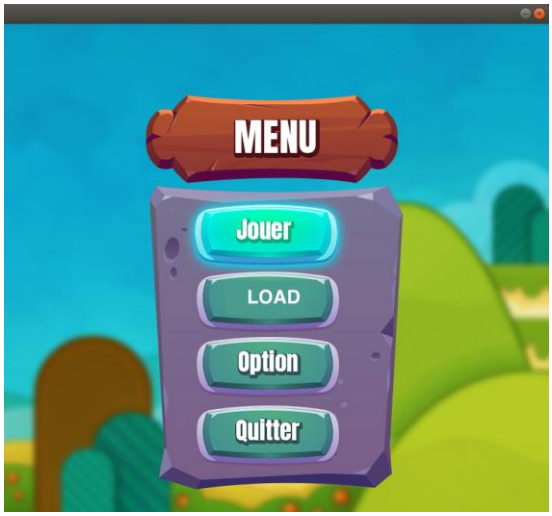

Figure 3 :Menu principale

Pour pouvoir restaurer le jeu nous devons faire des modifications sur le main au niveau de la fonction de mise à jour. Si l'utilisateur tape sur bouton load, nous devons appeler la fonction **load_game**. Cette fonction doit modifier le contenu des variables de base du jeu comme la position du joueur , camera , background , etc… Ainsi lors de l’affichage du jeu, au lieu d’afficher le joueur à sa position initiale , il sera automatiquement affiché à la position définie par la fonction **load_game** ainsi de suite pour les autres variables.

3. Travail à faire

Implémenter les fonctions précédemment exposées afin de rajouter l’option de sauvegarde et chargement du jeu.